



Universität Hamburg
DER FORSCHUNG | DER LEHRE | DER BILDUNG

Projektdokumentation

„TETRIS“

Fachbereich Informatik

64-146a Praktikum Programmiertechnisches Praktikum

Sommersemester 2025

Andreas Heymann Dipl.-Inform.

Abgabe: 17.08.2025

Vorname, Name	Batuhan Can	Torben Haase
E-Mail	batuhan.can@studium.uni-hamburg.de	torben.haase-1@studium.uni-hamburg.de
Matrikelnummer	7773734	7610371
Studiengang	Wirtschaftsinformatik (Bc.S)	Informatik (Bc.S)
Fachsemester	4	6
PTP Gruppe	mi02	mi02

Abstract

Dieses Projekt beschreibt die Umsetzung des weltbekannten Spiels **Tetris** in Java, entwickelt im Jahr 2025 im Rahmen des Moduls „*Praktikum Programmiertechnisches Praktikum*“. Ziel war es, in einem Zweierteam das klassische Tetris nachzubilden, eigene Funktionen zu ergänzen, sich mit typischen Anforderungen und Herausforderungen der Softwareentwicklung auseinanderzusetzen sowie die Kenntnisse in der GUI-Programmierung zu vertiefen.

Das Spiel ermöglicht es den Nutzern, herabfallende Blöcke zu steuern und vollständige Reihen zu bilden, die anschließend gelöscht werden. Die Realisierung nutzte gängige Werkzeuge zur GUI-Programmierung und Audiounterstützung, wobei besonderes Augenmerk auf Steuerung, visuelle Klarheit, Kollisionserkennung, Spiellogik und flüssige Animationen gelegt wurde.

Das fertige Programm bietet unter anderem Punktwertung, Levelaufstieg, steigenden Schwierigkeitsgrad, Hintergrundmusik, Animationen, eine Highscore-Tabelle und eine intuitive Steuerung. Zudem verdeutlicht das Projekt die praktische Anwendung von OOP-Prinzipien, der MVC-Architektur, JUnit-Tests sowie den Einsatz relevanter Java-Bibliotheken.

Inhaltsverzeichnis

Projektidee	2
Anforderungsdefinition	2
User Stories (mit Akzeptanzkriterien)	4
Mockup	5
CRC-Cards für ausgewählte zentrale Klassen.....	6
Grobbeschreibung für das Gesamtprojekt	7
Aufteilung / Modularisierung / Programmstruktur	7
Prioritätenliste.....	8
Projektplan.....	9
UML-Diagramme	10
Testplan – Tetris (JUnit 5)	11
Stand des Projektes	15
Funktionsbeschreibung aus Nutzersicht	17
Anlage I: Architekturübersicht des Systems (Grobes UML-Diagramm)	18
Anlage II: Detaillierte Klassendiagramme zur Modellierung der Spiellogik	19

Projektidee

Entwicklung einer eigenen Variante des etablierten Spielklassikers Tetris als Projektteil im Rahmen des Programmiertechnischen Praktikums. Desktop-Anwendung mit klarer MVC-Trennung, 60 FPS, lokal speicherbaren Highscores und optionaler Musik. Die Logik umfasst 7-Bag-Zufall, SRS-Wall-Kicks und ein Ghost-Piece. Die GUI basiert auf Swing mit skalierbarem Layout. Highscores werden im Benutzerverzeichnis dauerhaft gespeichert.

Ziele/Nutzen

- **Spielspaß:** klassisches Tetris
- **Architektur & Didaktik:** klare Struktur, testbare Kernlogik.

Abgrenzung (Nicht-Ziele)

- **Kein** Online-Multiplayer/Netzwerk-Highscore.
- **Kein** Gamepad-/Controller-Support.

Stakeholder

- **Spieler:** schneller Start, einfache Steuerung.
- **Entwickelnde/Lehrende:** Anwendung des im ersten Projektteil erworbenen Wissens. Nachweis eines eigenständig entwickelten Projekts im Zweier-Team unter Einhaltung der gegebenen Rahmenbedingungen (siehe PTP2025 Website).

Anforderungsdefinition

Funktionale Anforderungen

- **Spiel-Loop & Steuerung:** ~60 FPS; Eingaben: $\leftarrow/\rightarrow$, $\downarrow$, $\uparrow$, Leertaste (Hard Drop), P (Pause), ENTER (Start/Restart).
- **Tetromino-Logik:** 7-Bag-Zufall, SRS-Wall-Kicks, Ghost-Vorschau, "Next"-Anzeige.
- **Linien & Scoring:** Erkennung voller Reihen; Punkte = $100 * \text{Level} * \text{gelöschte Linien}$; Level-Up alle 10 Linien; Gravitation schneller mit Level.
- **Zustände:** Startscreen (Logo-Animation), Playing, Game Over (Overlay).
- **Highscores:** Name (≤ 15 Zeichen) + Score; Abfrage bei Qualifikation; absteigend sortiert; Limit N; Speicherung im Benutzerverzeichnis.

- **Audio:** Start- und Gameplay-Musik als Loop; an/aus schaltbar.
- **Hilfe:** Dialog mit Tastenbelegung.
- **UI-Skalierung:** Virtuelle Fläche; Buttons per virtuelle Koordinaten positioniert.

Nicht-funktionale Anforderungen

- **Performance:** stabile Darstellung; keine spürbaren Ruckler.
- **Portabilität:** Windows/macOS/Linux, Java ≥17, Swing.
- **Robustheit:** Fehlerhafte Highscore-Zeilen werden ignoriert; Speichern atomar (Temp-Datei + Move) mit Fallback.
- **Testbarkeit:** Unit-Tests für Kernbereiche (Seven-Bag, Qualifikation, Line-Clear, Scoring).
- **Usability:** klare Farben, gut lesbare Schrift, Ghost halbtransparent.
- **Basis-Barrierefreiheit:** komplette Bedienung per Tastatur.

Randbedingungen & Systemkontext

- **Laufzeitumgebung:** JVM 17+, JLayer für MP3
- **Dateisystem:** `~/.ptp-tetris/highscores.txt` (ohne Admin-Rechte).
- **Assets:** nur eigene (selbsterstelltes Tetris Logo) freie Medien (beide Musikstücke unterliegen der Pixabay-Inhaltslizenz und sind somit auch ohne die Nennung des Autors frei zu nutzen)

Drittanbieter-Lizenzen

JavaZOOM: JLayer — MP3 Decoder

Beschreibung: Bibliothek zum Dekodieren von MP3-Musik. *Optionale Abhängigkeit.*

Rechtliches

- Copyright: © 1999–2008 JavaZOOM
- Lizenz: GNU LESSER GENERAL PUBLIC LICENSE, Version 2.1 (LGPL 2.1)

User Stories (mit Akzeptanzkriterien)

Spielerfahrung

- *Als Spieler* starte ich mit ENTER, *damit* ich sofort spielen kann.
=> ENTER auf Startscreen → Status "Playing", Musik wechselt, erster Snapshot sichtbar.
- *Als Spieler* bewege/rotiere ich den Stein und nutze Hard Drop, *damit* ich präzise platziere.
=> </> bewegen; ↑ rotiert (inkl. Wall-Kick); SPACE = Hard Drop.
- *Als Spieler* sehe ich ein Ghost-Piece, *damit* ich die Landeposition erkenne.
=> Ghost unten, halbtransparent, kollisionsfrei.
- *Als Spieler* sehe ich "Next", *damit* ich plane.
=> Vorschau rechts, zentriert im Fenster.
- *Als Spieler* erhalte ich Punkte und Level-Ups, *damit* die Herausforderung steigt.
=> Formel/Level-Up wie definiert; Gravitation spürbar schneller.
- *Als Spieler* pausiere ich das Spiel, *damit* ich unterbrechen kann.
=> P toggelt Pause; Overlay "PAUSED"; Tick stoppt.

Highscore Erfahrung

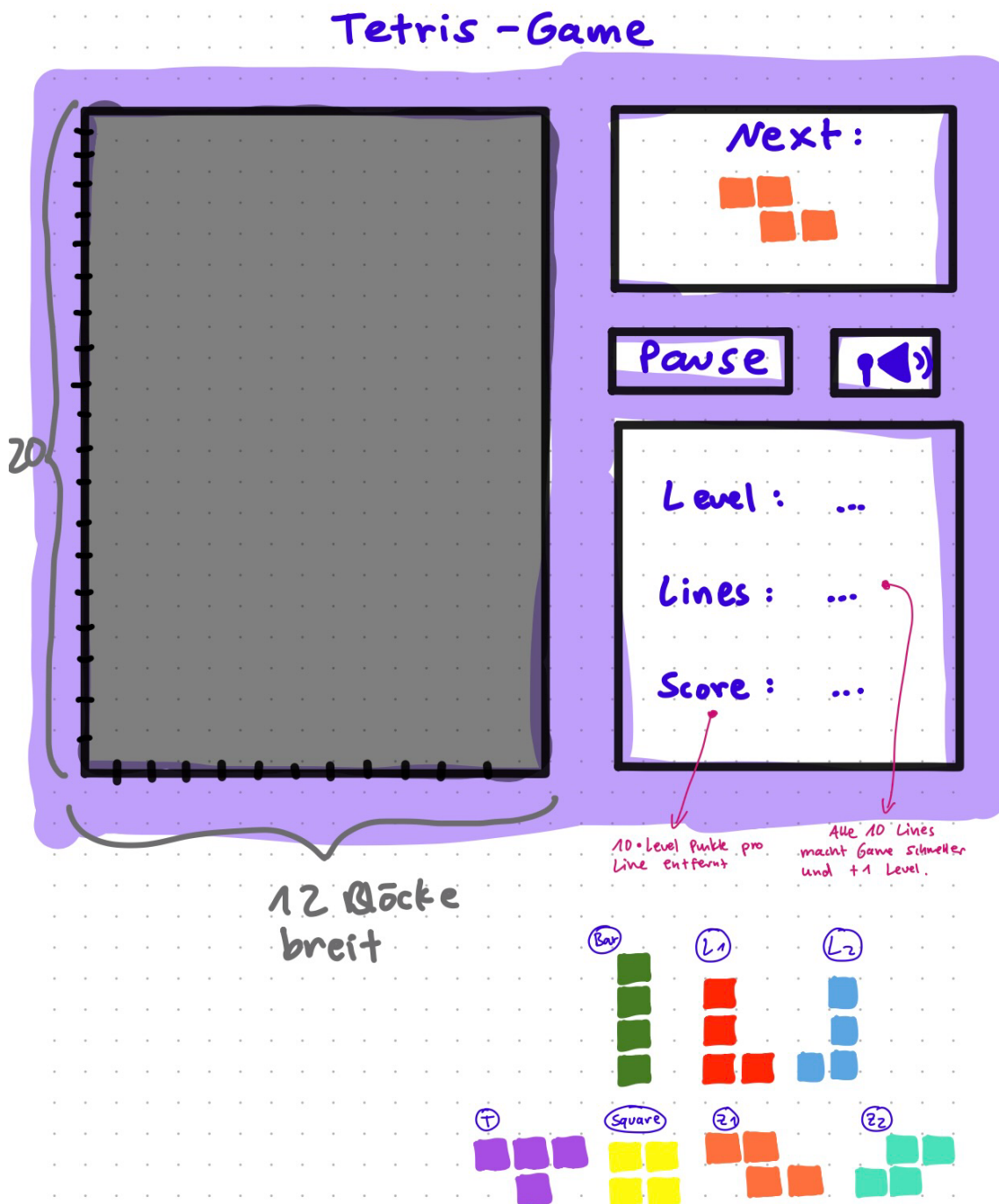
- *Als Spieler* gebe ich meinen Namen bei Qualifikation ein, *damit* ich im Ranking erscheine.
=> Dialog bei Qualifikation; Validierung ≤15; leere Eingabe wird ignoriert.
- *Als Spieler* sehe ich die Top-Einträge, *damit* ich mich vergleichen kann.
=> Anzeige Top-5

Audioerfahrung und Hilfsangebot

- *Als Spieler* schalte ich Musik an/aus, *damit* ich ungestört bin.
=> Button toggelt; Track wechselt je Zustand.
- *Als Spieler* öffne ich eine kurze Hilfe, *damit* ich die Steuerung lerne.
=> Dialog listet Tasten; aus dem Spiel heraus wird bei geöffnetem Dialog pausiert.

Mockup

In der ersten Arbeitswoche erstellten wir ein Mockup, das in der finalen Umsetzung nahezu vollständig übernommen wurde. Lediglich der ursprüngliche Titel *Tetris-Game* wurde entfernt und durch ein eigens gestaltetes Tetris-Logo oberhalb der Next-Box ersetzt. Außerdem kam ein neuer Hilfs-Button hinzu, und die Spielfläche wurde in der Breite von 12 auf 10 Spalten reduziert.



CRC-Cards für ausgewählte zentrale Klassen

GameController — steuert Tick/Eingaben; registriert Listener und publiziert Snapshots.

Collaborators: GameModel, PlayService (aktuelle Instanz), Snapshot-Listener
(z. B. GameFrame/BoardPanel).

PlayManager (implements PlayService) — Bewegung/Rotation/Drop, Kollision, Line-Clear, Scoring/Level/Gravitation, Ghost/Next, Snapshots.

Collaborators: MinoProvider, ScoringPolicy, WallKick, SevenBagRandom.

GameModel — hält und wechselt den aktuellen PlayService (für Restart).

Collaborators: PlayService, GameController.

BoardPanel — Rendering (Feld/Sidebar/Overlays), UI-Skalierung, Highscore-Anzeige, Partikel-Effekte; empfängt Snapshots.

Collaborators: GameController (Snapshot-Feed), TetrisRenderer, StartScreenAnimator, Overlays.

TetrisRenderer — zeichnet Feld, Blöcke, Ghost, Next, Stats.

Collaborators: GameSnapshot/BlockDTO, Theme, BoardPanel (Geometrie-Konstanten).

GameFrame — Fenster, Buttons, Key-Bindings, Musik-Kontext, Start/Restart, Highscore-Flow.

Collaborators: GameController, BoardPanel, AudioService, HighscoreFlow, view.input.KeyBindings.

HighscoreService — Laden/Sortieren; Qualifikation prüfen; Einfügen (Namens-Truncation) und Speichern.

Collaborators: HighscoreRepository.

FileHighscoreRepository (implements HighscoreRepository) — Datei-I/O (lesen/schreiben), robust (atomare Speicherung).

Collaborators: JDK-I/O (java.nio.file.*); verwendet von HighscoreService.

HighscoreFlow (interface) — definiert Highscore-Anwendungsfall (Laden, Game-Over-Verarbeitung).

Collaborators: HighscoreService, NamePrompt.

DefaultHighscoreFlow (implements HighscoreFlow) — verwaltet Game-Over: Qualifikation prüfen → Name abfragen → speichern → Liste zurückgeben.

Collaborators: HighscoreService, NamePrompt.

AudioService / JLayerAudioService — MP3-Loop abspielen, Stop/Start, Quelle wechseln.

Collaborators: GameFrame (Steuerung); externe Lib: JavaZoom **JLayer**.

SwingNamePrompt (implements NamePrompt) — Namenseingabe ≤15, Validierung/Fehlermeldung.

Collaborators: HighscoreFlow (Aufrufer), JOptionPane.

Grobbeschreibung für das Gesamtprojekt

Es wird ein klassisches **Singleplayer-Tetris** mit Java und Swing entwickelt. Die Spiellogik umfasst 7-Bag-Zufall, SRS-Rotationen mit Wallkicks sowie Scoring, Level und Gravitation. Eine grafische Oberfläche stellt Spielfeld, Ghost-Mino, Next-Vorschau, Start- und Game-Over-Screens dar. Highscores werden dauerhaft gespeichert, Hintergrundmusik und Partikeleffekte runden das Spiel ab.

Aufteilung / Modularisierung / Programmstruktur

- **main**
 - Einstiegspunkt, Bootstrapping, Wiring von MVC + Services
- **app**
 - HighscoreService (Orchestrierung von Repository + Qualifikation)
- **audio**
 - AudioService-Port und JLayer-Adapter für Musiksteuerung
- **controller**
 - GameController (Loop, Input, Snapshot-Events)
 - HighscoreFlow (NamePrompt, Qualifizierung, Speicherung)
- **model**
 - Spiellogik: PlayManager/Service, Mino-Klassen, Scoring, SevenBagRandom, Wallkicks
 - Snapshot-DTOs (immutable)
 - HighscoreRepository (Interface + File-Adapter)
- **view**
 - Swing-UI: GameFrame, BoardPanel, Renderer, Overlays, Themes
 - Start-Screen Animation, NamePrompt, KeyBindings

Prioritätenliste

Must

- **Kernspiel:** Die grundlegende Spiellogik wie Bewegung, Rotation nach SRS mit Wallkicks, Kollisionserkennung und das Löschen kompletter Linien.
- **Scoring/Level/Gravitation:** Punktevergabe, Levelaufstieg und die damit verbundene Beschleunigung der Fallgeschwindigkeit.
- **Ghost-Mino und Next-Vorschau:** Ein GhostMino zeigt die zukünftige Position an, während ein Vorschaubereich den nächsten Block darstellt.
- **Highscores:** Spielergebnisse werden dauerhaft gespeichert und über ein Namenseingabefeld in die Highscore-Liste aufgenommen.
- **Start- und Game-Over-Screens:** Das Spiel verfügt über einen animierten Startbildschirm sowie ein Overlay am Ende einer Partie.
- **Audio-Loop mit Toggle:** Hintergrundmusik läuft in einer Schleife und kann während des Spiels an- oder ausgeschaltet werden.
- **Spielbar als Runnable JAR:** Das fertige Programm lässt sich plattformunabhängig starten und direkt ausführen.

Should

- **Partikel-Effekte bei Line-Clear:** Beim Löschen von Reihen erscheinen visuelle Effekte, die das Spielerlebnis lebendiger gestalten.
- **Tooltips und UI-Feinschliff:** Kleine Hilfetexte sowie optische Verbesserungen in der Benutzeroberfläche erhöhen die Bedienbarkeit.
- **Basis-Tests:** Wesentliche Funktionalitäten wie Scoring, Highscore-Logik und Bag-Randomizer sind durch einfache Unit-Tests abgesichert.

Nice-to-have

- **Konfigurierbare Themes:** Farbgestaltung / Anpassung des Designs des Spiels
- **Rebindable Keys:** Individuelle Neubelegung der Steuerungstasten
- **Highscore-Import/Export:** Highscore-Listen ließen sich zwischen verschiedenen Installationen austauschen oder sichern.

Projektplan

PTP – Aufgabe 2: Projektbeschreibung (Sollkonzept)

Name(n): mi02 – Batuhan Can, Torben Haase

Datum: Erste Version 04.06.2025, Letzte Aktualisierung 17.08.2025

LV, Aufgabe: PTP – Aufgabe 2

Ziel

Ein spielbares Tetris in Java/Swing nach bekannten Regeln mit Highscores, SRS-Rotationen und Audio.

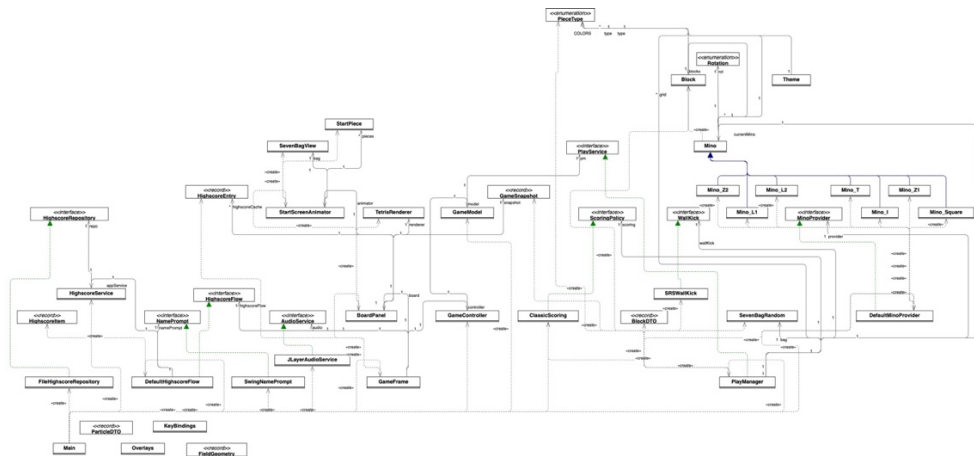
User Stories

Prio	User Story	Zeitaufwand (SOLL)	Zeitaufwand (IST)
1	Spiel beginnen (Startscreen, Gamescreen)	3h	8h
2	Stein bewegen & rotieren	6h	10h
3	Linien löschen + Scoring/Level	5h	7h
4	Ghost-Mino & Next- Vorschau	4h	8h
5	Spiel beenden / Game-Over- Overlay	2h	4h
6	Highscore speichern	4h	7h
7	Audio-Loops	3h	5h
8	Partikel-Effekte bei Clear	3h	4h
9	Tests	5h	7h
10	Refactoring	2h	10h
Summe		48h	70h

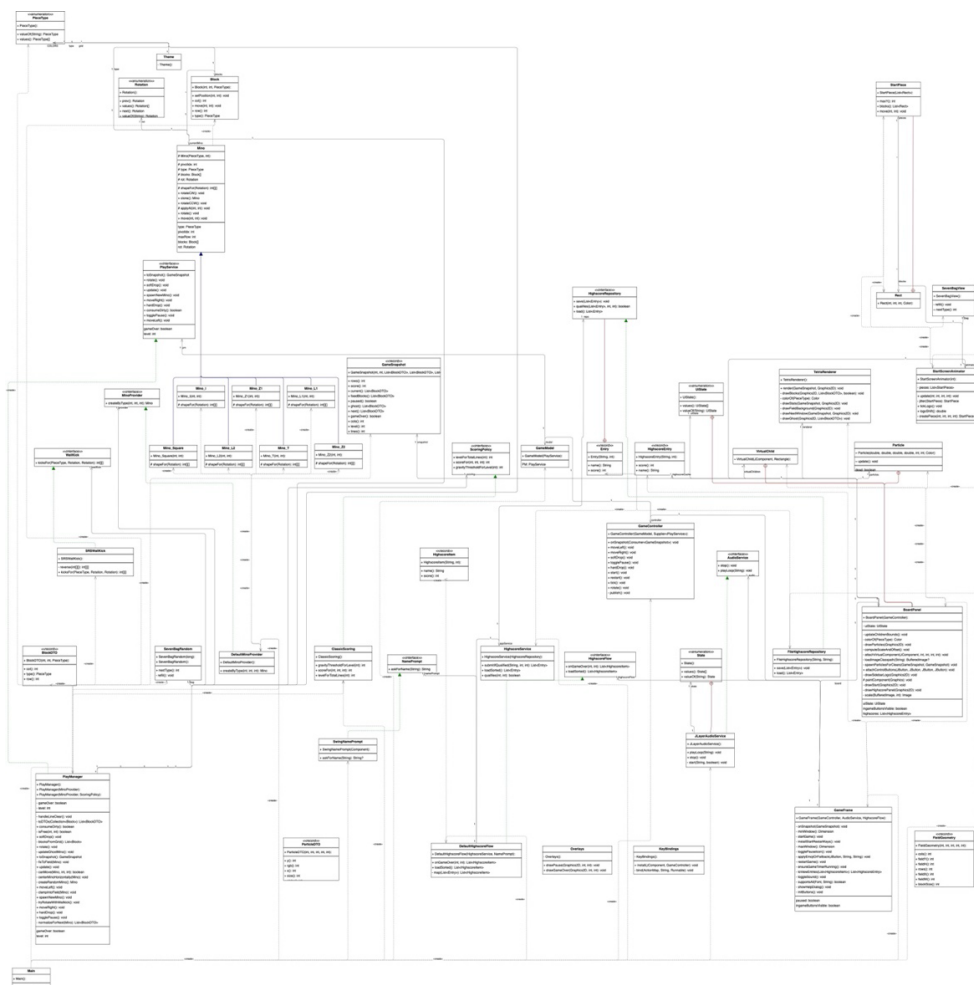
UML-Diagramme

Die UML-Diagramme wurden in **IntelliJ IDEA** mit dem *Diagrams*-Plugin erstellt und anschließend in **diagrams.net** grafisch aufbereitet. Aufgrund ihres Umfangs sind sie hier nur verkleinert dargestellt; die vollständigen Versionen finden sich im Anhang.

Architekturübersicht des Systems (Grobes UML-Diagramm)



Detaillierte Klassendiagramme zur Modellierung der Spiellogik



Testplan – Tetris (JUnit 5)

Zielsetzung

- Ziel dieses Testplans ist es, die **Korrektheit der Kernlogik** des Tetris-Projekts systematisch zu prüfen.
- Im Fokus stehen: Highscore-Verwaltung, Spiellogik (Scoring, Level, Zufall, Rotationen), sowie Controller-Fluss.
- GUI-Rendering und reale Audiowiedergabe sind **nicht** Gegenstand der Unit-Tests.

Testumfang (In Scope / Out of Scope)

In Scope

- **Highscores**
 - HighscoreService, HighscoreRepository (Default-Qualify), FileHighscoreRepository
 - DefaultHighscoreFlow (Nutzerfluss ohne UI)
- **Game Core**
 - PlayManager (über PlayService-API)
 - ClassicScoring (Punkte/Level/Gravitation)
 - SRSWallKick (Kick-Tabellen inkl. Reverse)
 - SevenBagRandom (7-Bag-Eigenschaften)
- **Controller**
 - GameController (Delegation, Publish-Verhalten)

Out of Scope

- **View/Rendering** (BoardPanel, TetrisRenderer, Overlays, Startanimation)
- **Echte Audioausgabe** (JLayerAudioService) – optional nur Smoke-Tests
- **Externe Ressourcen** (Bilder/MP3-Dateien)

Teststrategie & -methodik

- **Framework:** JUnit 5
- **Ebenen**
 - **Unit-Tests** für reine Logikkomponenten
 - **Gezielte Integrations-Tests** für Datei-Persistenz mittels temporären Verzeichnisses
- **Isolation:**
 - Für Unit-Tests werden **Testdoubles** im Testcode verwendet (RAM statt Datei, kein Dialog/Audio).
 - Für `FileHighscoreRepository` werden echte Dateien nur in einem **temporären Home** getestet.
- **Determinismus:**
 - 7-Bag-Tests mit fixem Seed.
 - Keine Abhängigkeit von Timern/echtem UI.

Testdaten & Testdoubles

- **InMemory-HighscoreRepository (Testdouble)**
 - **Zweck:** Unit-Tests für `HighscoreService` und `DefaultHighscoreFlow` ohne Dateizugriff.
 - **Verhalten:** Verwaltet eine `List<Entry>` im RAM und simuliert Speichern/Laden synchron.
- **FakeNamePrompt (Testdouble)**
 - **Zweck:** Vermeidet UI-Dialoge im Test; liefert vorgegebene Namen (inkl. leer/`null`) für Highscore-Einträge
- **FakePlayService (Testdouble)**
 - **Zweck:** Dient zur isolierten Prüfung des `GameController`. Zählt aufgerufene Methoden und liefert kontrollierte Snapshots zurück.
- **DeterministicMinoProvider (Testdouble)**
 - **Zweck:** Für gezielte Tests des `PlayManager` werden bestimmte Formen (z. B. nur I-Steine) bereitgestellt, um Linien-Clears deterministisch zu prüfen.

Testdoubles werden ausschließlich in den Testklassen verwendet!

Testfälle (Übersicht & Akzeptanzkriterien)

ClassicScoring

- **Score-Formel:** Überprüfung, dass die Punkteformel korrekt berechnet wird (z. B. 1–4 Linien in Level 1).
- **Level-Fortschritt:** Sicherstellen, dass der Level alle 10 Linien ansteigt.
- **Gravitation-Minimum:** Kontrolle, dass die Fallgeschwindigkeit nie unter einem Mindestwert liegt.

SevenBagRandom

- **7 eindeutig:** Erste 7 Steine decken immer alle Typen ab.
- **Refill:** Nach 14 Steinen entstehen zwei vollständige Permutationen.
- **Seed-Determinismus:** Mit gleichem Seed entsteht dieselbe Sequenz.

SRSWallKick

- **O-Mino:** Es existieren keine Kicks außer der Startposition.
- **I 0→R:** Korrekte Anwendung der I-Mino-Kick-Tabelle.
- **JLSTZ 0→R:** Korrekte Anwendung der Standard-Kick-Tabelle für JLSTZ-Steine.
- **Reverse:** Rückwärtsrotation ist die exakte Negation der Vorwärts-Kicks.

HighscoreRepository

- **Unter Kapazität:** Jeder Score wird akzeptiert.
- **Voll & größer:** Ein höherer Score qualifiziert sich.
- **Voll & gleich:** Ein gleich hoher Score qualifiziert sich nicht.
- **Voll & kleiner:** Ein niedrigerer Score qualifiziert sich nicht.

FileHighscoreRepository

- **Save/Load & Sortierung:** Scores werden korrekt gespeichert, geladen und sortiert.
- **Semikolon-Sanitizing:** Namen mit ; werden zu , umgewandelt.
- **Robustheit:** Ungültige Zeilen führen nicht zu Fehlern, sondern werden ignoriert.
- **Vollständigkeit:** Gespeicherte Dateien enthalten alle Einträge vollständig.

HighscoreService

- **loadSorted ():** Gibt stets eine sortierte, unveränderliche Liste zurück.
- **qualifies ():** Korrekte Entscheidung, ob ein Score aufgenommen wird.
- **Leerer/Null-Name:** Ein leerer Name wird nicht gespeichert.
- **Namenskürzung:** Zu lange Namen werden auf 15 Zeichen gekürzt.
- **Kappung:** Liste überschreitet nie die maximale Länge.
- **Persistenz:** Nach erfolgreichem Einfügen wird gespeichert.

DefaultHighscoreFlow

- **Mapping:** Geladene Einträge werden korrekt ins DTO übertragen.
- **GameOver ohne Qualifikation:** Keine Namensabfrage, nur Highscore-Anzeige.
- **GameOver mit Qualifikation:** Namensabfrage, Speicherung und Liste wird aktualisiert.

PlayManager

- **Spawn/Ghost:** Nach dem Start ist ein Tetromino und sein Ghost korrekt platziert.
- **Gravitation:** Steine fallen nach der festgelegten Schwelle automatisch.
- **Grenzen:** Bewegungen bleiben innerhalb des Spielfelds.
- **Rotation:** Rotationen funktionieren korrekt an den Wänden mit Wall-Kick.
- **Hard-Drop:** Steine fixieren sich sofort und ein neues Tetromino erscheint.
- **Linienlöschung:** Volle Reihen werden entfernt, Score/Level angepasst.
- **Game Over:** Bei Spawn-Kollision endet das Spiel.
- **Pause:** Im Pausenmodus erfolgen keine Bewegungen.
- **Dirty-Flag:** Änderungen setzen die Dirty-Flag korrekt und zuverlässig zurück.

GameController

- **Start:** Ein Spielstart erzeugt einen Snapshot und spawnnt ein neues Tetromino.
- **Tick/Dirty:** Nur bei Änderungen werden Snapshots veröffentlicht.
- **Eingaben:** Steuerbefehle werden korrekt weitergeleitet und publiziert.
- **Restart:** Neustart ersetzt den PlayService und setzt das Spiel zurück.

Stand des Projektes

Was ist fertig?

Das Projekt umfasst ein vollständig spielbares **Singleplayer-Tetris** mit stabiler Spiellogik (7-Bag, SRS-Rotationen, Scoring/Level/Gravitation, Linien-Clears) sowie Ghost-Mino, Next-Vorschau, Start- und Game-Over-Screens, Pausenfunktion, Partikel-Effekten und einer Highscore-Liste mit Namenseingabe und Speicherung.

- **Startfläche:** Vor Spielbeginn erscheint eine Startfläche mit Logo, Animationen und Hintergrundmusik. Mit der *Enter*-Taste startet das Spiel.
- **Spielfläche:** Matrix von 20×10 Feldern, in der Minos bewegt und rotiert werden können. Wände sind undurchlässig; Wallkicks korrigieren Rotationen. Bei Überfüllung erscheint *Game Over* mit Restart-Button.
- **Highscore-Tafel:** Nach Spielende kann der Score mit einem Spielernamen (max. 15 Zeichen) gespeichert werden. Die Top-5 mit Platzierung, Name und Score werden absteigend gelistet, Platz 1 erhält eine Krone.
- **Next-Feld:** Zeigt den nächsten zufälligen Mino an, sodass sich der Spieler vorbereiten kann.
- **Lines, Score, Level:** Anzeige der gelöschten Reihen, des aktuellen Scores und des Levels. Mit steigendem Level erhöht sich das Tempo der Minos.
- **Pause, Musik, Hilfe:** Buttons ermöglichen Pausieren, Musik an/aus und das Anzeigen einer Steuerungshilfe (pausiert automatisch).
- **Ghost-Mino:** Ein transparenter Schattenstein zeigt die voraussichtliche Landeposition.
- **Hintergrundmusik:** Ruhige Musik auf der Startfläche, schnellere im Spiel – beide angelehnt an die Tetris-Melodie.
- **Steuerung:** Minos über Pfeiltasten, Hard-Drop per *Leertaste*.
- **Schwierigkeitsgrad:** Steigende Herausforderung mit jedem Level.
- **Animationen:** Lösch- und Restart-Animationen steigern die visuelle Wirkung.

Was wurde weggelassen/verworfen (warum)?

- **Dualmodus:** Zwei Spielfelder auf einer Tastatur waren geplant, wurden aber aus Zeitgründen gestrichen.
- **Mehrspieler:** Ein Online- oder Netzwerkmodus wurde nicht umgesetzt. Stattdessen sorgt die Highscore-Tabelle für Vergleichsmöglichkeiten am selben Gerät.
- **Alternative Fallmechanik & eigene Wallkick-Logik:** Im Projektverlauf verworfen.
- **Maussteuerung:** Die Idee, Minos auch per Maus zu steuern, wurde zugunsten einer reinen Tastatursteuerung verworfen, da diese als intuitiver und passender für Tetris empfunden wurde.
- **Weitere Erweiterungen:** Multiplayer, KI-Gegner oder konfigurierbare Themes wurden bewusst zurückgestellt.

Wo gibt es „Macken“?

- Musik auf dem Startbildschirm läuft erst einige Sekunden nach Spielstart
- Die drei Buttons sind kurz sichtbar, bevor der GameScreen erscheint.
- Nach 2x Betätigen des Musik-Pause-Buttons startet das Stück von vorne.
- Es gibt keine benutzerfreundliche Möglichkeit, die Highscore-Liste zurückzusetzen.

Wie war die „Workload“ tatsächlich?

Der Start verlief eher langsam, die letzten beiden Wochen waren von intensiver Arbeit geprägt, wodurch ein Großteil der Funktionalität nachgezogen wurde. Insgesamt belief sich der Aufwand auf ca. **70 Stunden** für die Entwicklung, **ohne** den zusätzlichen Zeitaufwand für die Erstellung und Fertigstellung der Projektdokumentation.

Aufteilung der Arbeit in der Gruppe

Eine feste Aufgabentrennung wurde nicht vorgenommen. Wir haben in allen Abschnitten **gemeinsam** gearbeitet, diskutiert und implementiert. Eine klare Zuweisung von Verantwortlichkeiten ist daher nicht mehr möglich und im Sinne einer differenzierten Bewertung aus unserer Sicht nicht sinnvoll.

Funktionsbeschreibung aus Nutzersicht

Der Nutzer erhält analog zur Abgabe eine ausführbare JAR-Datei (tetris-game.jar).

Somit kann das Spiel ohne DIE/Build-Tool gestartet, eine Java-Laufzeitumgebung ist jedoch erforderlich.

Auf Class-Build und Classpath wird am Ende dieses Kapitels eingegangen.

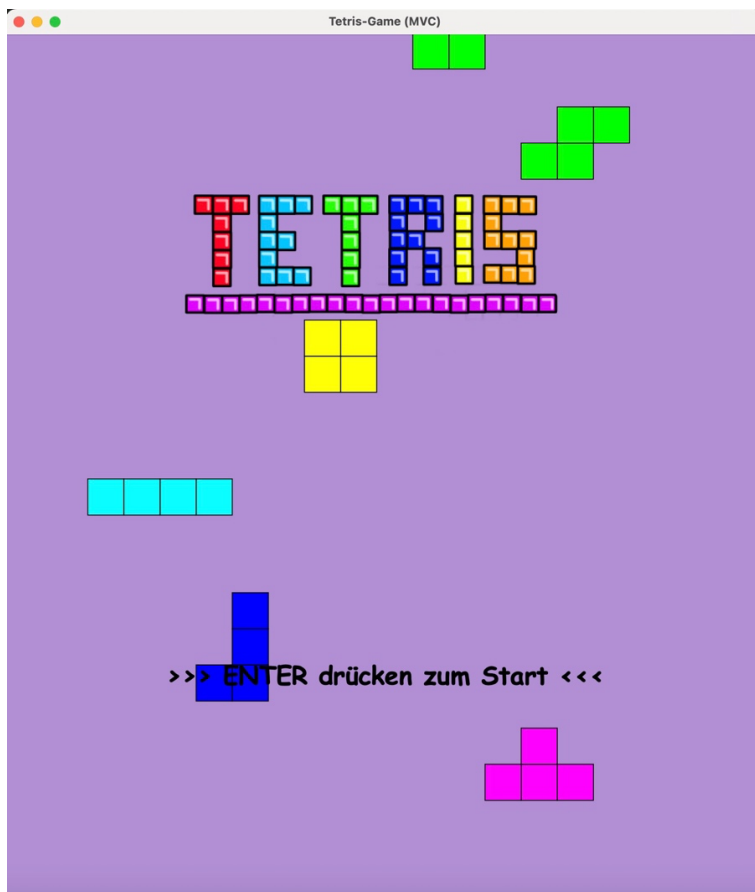
Programmstart

Voraussetzung: Java 17 oder höher.

Starten:

- Doppelklick auf tetris-game.jar, oder
- über Konsole: `java -jar tetris-game.jar`

Erster Bildschirm: Startscreen mit animierten Steinen. Zum Start ist ENTER zu drücken



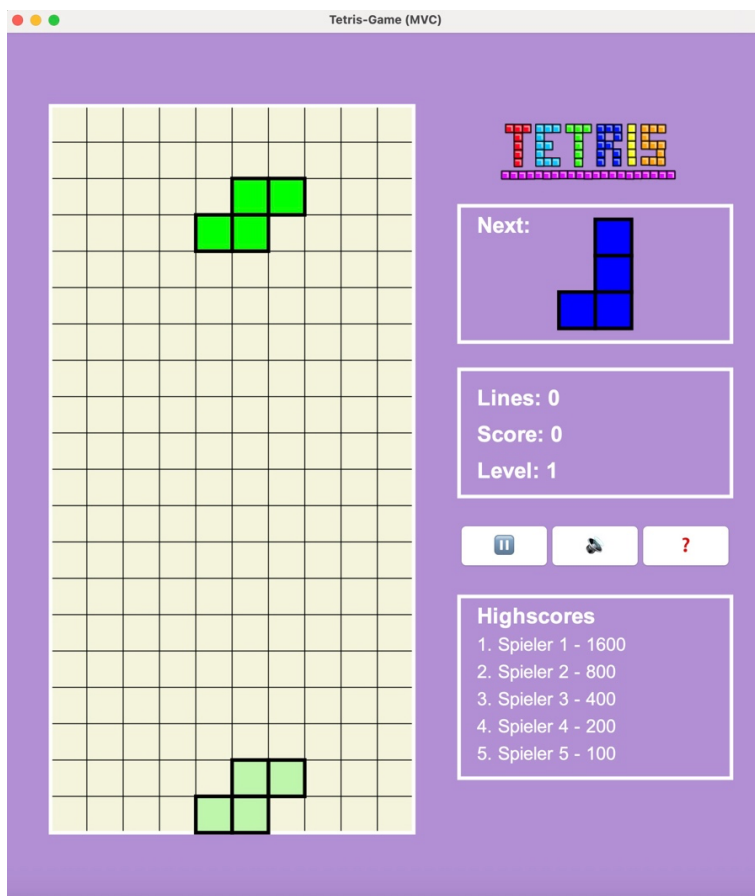
Vorgehensweise nach dem Start

Standard-Vorgehen

1. Der Nutzer drückt ENTER – das Spiel startet.
2. Steine werden mit den Pfeiltasten/Leertaste platziert (vgl. Steuerungstabelle).
3. Reihen schließen, um Punkte zu sammeln.
4. Game Over: Bei Qualifikation gibt der Nutzer einen Namen (maximal 15 Zeichen) ein;
5. Möglichkeit ein neues Spiel zu beginnen über RESTART/ENTER
6. Beenden des Spiels durch Schließen des Fensters (X) (unter Windows z. B. Alt+F4, unter macOS ⌘Q). Laufende Spiele gehen dabei verloren.

Alternativen & Bedienhilfen

- Buttons rechts: Pause / Sound / Hilfe / Restart (nach Game Over).
- Sound jederzeit umschaltbar; Musik läuft im Loop.
- Pause (P) friert das Spiel ein (kein Punktverlust).

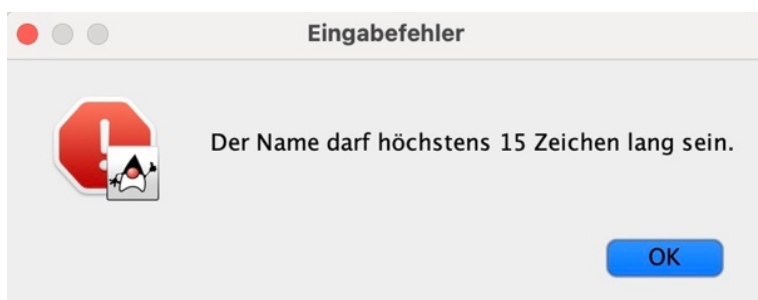


Steuerung (auf einen Blick)

Taste	Funktion
← / →	Nach links / rechts bewegen
↓	Soft Drop (schneller fallen)
↑	Rotieren
Leertaste	Hard Drop (sofort platzieren)
P	Pause umschalten
ENTER	Start / Restart

Eingaben & Wertebereiche

- **Spielername:** maximal 15 Zeichen; leere/Whitespace-Namen werden ignoriert; „;“ wird zu „,“ normalisiert. Bei Überschreitung erscheint eine Fehlermeldung.
- **Highscores:** TOP 5 werden angezeigt; TOP 10 werden lokal gespeichert (absteigend). Speicherort: ~/.ptp-tetris/highscores.txt.
- **Highscores zurücksetzen:**
 - Empfohlen: Zuerst das Spiel beenden.
 - Manuelles Löschen der Datei über Konsole
 - Ergebnis: Die Highscore-Liste ist leer; beim nächsten Eintrag wird die Datei neu angelegt.
- **Score/Level/Lines:** werden intern berechnet (keine direkte Eingabe).



Häufige Probleme & Lösungen

Problem	Ursache	Lösung
JAR startet nicht per Doppelklick	Java nicht installiert / Dateiverknüpfung fehlt	Java 17+ installieren; alternativ Konsole: java -jar tetris-game.jar
Kein Sound	Ressourcen stumm/aus, Ausgabegerät	Sound-Button aktivieren; System-Audio prüfen
Highscores fehlen	Keine Schreibrechte im Benutzerordner	In Umgebung mit Schreibrechten starten
Name länger als 15 Zeichen	Eingabegrenze aktiv	Fehlermeldung bestätigen; erneut eingeben (≤ 15)
Fenster zu groß/klein	Auflösung/Skalierung	Fenstergröße anpassen – UI skaliert automatisch

Optional – Class-Build & Classpath (für Entwicklung)

Für den regulären Nutzerbetrieb nicht erforderlich; nur relevant für Entwicklung/Lehre.

Ressourcendateien werden per absolutem Pfad (beginnend mit /) geladen,
z. B. /tetris-theme.mp3, /PTP Tetris-Logo.png.

Bei eigenem Build ist die JLayer-JAR in den Classpath aufzunehmen.

Hinweis: Für den Nutzerbetrieb genügt die JAR-Datei. Der Class-Build ist optional und dient ausschließlich Entwicklungszwecken.

